数据库设计与开发
Database Design & Development

计算机学院　单纯
sherryshan@bit.edu.cn
2024年3月

数据库恢复技术
Database Recovery Techniques

计算机学院　单纯
sherryshan@bit.edu.cn
2024年5月

内容概览（1）

- 10.1 事务的基本概念和特征
- 10.2 数据库恢复的必要性
- 10.3 数据库恢复策略
- 10.4 恢复的实现技术
- 10.5 数据转储与恢复

内容概览（2）

- 10.6 基于日志的数据库恢复
- 10.7 检查点恢复技术
- 10.8 数据库镜像恢复技术
- 10.9 小结

10.1 事务的基本概念和特征

- 10.1.1 事务的基本概念
- 10.1.2 事务特征
- 10.1.3 事务状态

10.1.1 事务的基本概念

一、什么是事务
二、如何定义事务

一、什么是事务

- 事务（Transaction）是用户定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的工作单位
- 事务和程序是两个概念
  - 在关系数据库中，一个事务可以是一条SQL语句，一组SQL语句或整个程序
  - 一个应用程序通常包含多个事务
- 事务是恢复和并发控制的基本单位

二、如何定义事务（1）

- 显式定义方式

BEGIN TRANSACTION	BEGIN TRANSACTION
SQL 语句1	SQL 语句1
SQL 语句2	SQL 语句2
… …	… …
COMMIT	ROLLBACK

- 隐式方式
  - 当用户没有显式地定义事务时，DBMS按缺省规定自动划分事务

二、如何定义事务（2）

- COMMIT
  - 事务正常结束
    - 提交事务的所有操作（读+更新）
    - 事务中所有对数据库的更新永久生效
- ROLLBACK
  - 事务异常终止
    - 事务运行的过程中发生了故障，不能继续执行，回滚事务的所有更新操作
    - 事务滚回到开始时的状态

10.1.2 事务特征（1）

- 事务的四个特征（简称ACID特性/准则）
  - 原子性（Atomicity）
    - 事务的原子性是指事务是数据库操作的逻辑工作单位，不可再分割。事务中的所有操作要么都执行，要么都不执行

10.1.2 事务特征（2）

- 事务的四个特征（简称ACID特性/准则）（续）
  - 一致性（Consistency）
    - 事务的一致性是指事务执行的结果必须从一个状态转换到另一个状态时保持一致
      - 所谓状态保持一致是指当事务提交成功时，保存一致性的结果。而当事务提交不成功时，如数据库系统运行过程中发生故障，某些事务操作尚未完成就被中断，数据库将处在不一致的状态，这种状态应该撤销

实例

- 银行转帐：从帐号A中取出一万元，存入帐号B
  - 定义一个事务，该事务包括两个操作

A	B
A=A-1	B=B+1

  - 这两个操作要么全做，要么全不做
    - 全做或者全不做，数据库都处于一致性状态
    - 如果只做一个操作，数据库就处于不一致性状态

10.1.2 事务特征（3）

- 事务的四个特征（简称ACID特性/准则）（续）
  - 隔离性（Isolation）
    - 事务的隔离性是指一个事务的执行不能影响到另一个事务，即一个事务的内部操作相对于外部事务是隔离的。因此，并发执行事务时事务间不能相互干扰

实例

- T_1的修改被T_2覆盖了

T_1	T_2
① 读A=16	
②	读A=16
③ A? A-1 写回A=15	
④	A? A-3 写回A=13

10.1.2 事务特征（4）

- 事务的四个特征（简称ACID特性/准则）（续）
  - 持久性（Durability）
    - 事务的持久性是指一旦事务提交成功，数据修改是永久的
    - 接下来的其他操作或故障不应该对其执行结果有任何影响

10.1.3 事务状态（1）

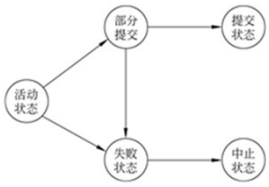
- 事务应该处在下列状态之一
  - 活动状态（active）
    - 初始状态，事务执行时事务处于活动状态
  - 部分提交状态（partially committed）
    - 事务最后一条语句被执行完毕后进入部分提交状态
    - 此时，事务中对数据的操作已经全部完成，但结果数据还驻留在内存中
    - 如果在此状态时，系统出现故障仍可能使事务不得不终止

10.1.3 事务状态（2）

- 事务应该处在下列状态之一（续）
  - 失败状态（failed）
    - 如果事务不能正常执行，事务就进入失败状态
    - 意味着事务没有成功地完成，必须回滚
    - 回滚（Rollback）就是撤销事务已经做出的任何数据更改
  - 中止状态（aborted）
    - 事务回滚并且数据库已经恢复到事务执行前的状态

10.1.3 事务状态（3）

- 事务应该处在下列状态之一（续）
  - 提交状态（committed）
    - 当数据库系统将事务中对数据的更改完全写入磁盘时，写入一条事务日志信息，标志着事务成功完成，这时事务就进入了提交状态



课堂练习

- 课堂练习10

10.2 数据库恢复的必要性（1）

- 故障是不可避免的
  - 计算机硬件故障
  - 系统软件和应用软件的错误
  - 操作员的失误
  - 恶意的破坏
- 故障的影响
  - 运行事务非正常中断
  - 破坏数据库

10.2 数据库恢复的必要性（2）

- 故障的种类
  - 系统故障
  - 事务内部的故障
  - 存储设备故障
  - 其他原因

10.2 数据库恢复的必要性（3）

- 数据库管理系统对故障的对策
  - DBMS提供恢复子系统
    - 保证故障发生后，能把数据库中的数据从错误状态恢复到某种逻辑一致的状态
    - 保证事务ACID
- 恢复技术是衡量系统优劣的重要指标

10.3 数据库恢复策略

- 10.3.1 事务故障的恢复
- 10.3.2 系统故障的恢复
- 10.3.3 介质故障的恢复

10.3.1 事务故障的恢复（1）

- 什么是事务故障
  - 某个事务在运行过程中由于种种原因未运行至正常终止点就夭折了
- 事务故障的常见原因
  - 输入数据有误
  - 运算溢出
  - 违反了某些完整性限制
  - 某些应用程序出错
  - 并行事务发生死锁

10.3.1 事务故障的恢复（2）

- 事务故障的恢复
  - 由恢复子系统利用日志文件撤消（UNDO）此事务已对数据库进行的修改
  - 事务故障的恢复由系统自动完成，不需要用户干预

10.3.1 事务故障的恢复（3）

- 事务故障的恢复步骤

  1. 反向扫描文件日志（即从最后向前扫描日志文件），查找该事务的更新操作

  2. 对该事务的更新操作执行逆操作。即将日志记录中“更新前的值”（Before Image, BI）写入数据库
     - 插入操作，“更新前的值”为空，则相当于做删除操作
     - 删除操作，“更新后的值”为空，则相当于做插入操作
     - 若是修改操作，则用BI 代替 AI（After Image）

10.3.1 事务故障的恢复（4）

- 事务故障的恢复步骤（续）

  3. 继续反向扫描日志文件，查找该事务的其他更新操作，并做同样处理

  4. 如此处理下去，直至读到此事务的开始标记，事务故障恢复就完成了

10.3.2 系统故障的恢复（1）

- 什么是系统故障
  - 整个系统的正常运行突然被破坏
  - 所有正在运行的事务都非正常终止
  - 内存中数据库缓冲区的信息全部丢失
  - 外部存储设备上的数据未受影响

10.3.2 系统故障的恢复（2）

- 系统故障的常见原因
  - 操作系统或DBMS代码错误
  - 操作员操作失误
  - 特定类型的硬件错误（如CPU故障）
  - 突然停电
- 系统故障造成数据库不一致状态的原因
  - 一些未完成事务对数据库的更新已写入数据库
  - 一些已提交事务对数据库的更新还留在缓冲区没来得及写入数据库

10.3.2 系统故障的恢复（3）

- 系统故障的恢复
  - Undo 故障发生时未完成的事务
  - Redo 已完成的事务
  - 系统故障的恢复由系统在重新启动时自动完成，不需要用户干预

10.3.2 系统故障的恢复（4）

- 系统故障的恢复步骤
  1. 正向扫描日志文件（即从头扫描日志文件）
     - Redo队列：在故障发生前已经提交的事务
       T_1，T_3, T_8…..
     - Undo队列：故障发生时尚未完成的事务
       T_2，T_4, T_5, T_6, T_7, T_9 ……

10.3.2 系统故障的恢复（5）

- 系统故障的恢复步骤（续）
  2. 对Undo队列事务进行UNDO处理
     - 反向扫描日志文件，对每个UNDO事务的更新操作执行逆操作
       T_2，T_4, T_5, T_6, T_7, T_9 ……
  3. 对Redo队列事务进行REDO处理
     - 正向扫描日志文件，对每个REDO事务重新执行登记的操作
       T_1，T_3, T_8…..

10.3.3 介质故障的恢复（1）

- 硬件故障使存储在外存中的数据部分丢失或全部丢失
- 介质故障比前两类故障的可能性小得多，但破坏性大得多

10.3.3 介质故障的恢复（2）

- 介质故障的常见原因
  - 硬件故障
  - 磁盘损坏
  - 磁头碰撞
  - 操作系统的某种潜在错误
  - 瞬时强磁场干扰

10.3.3 介质故障的恢复（3）

- 介质故障的恢复
  - 装入数据库发生介质故障前某个时刻的数据副本
  - 重做自此时始的所有成功事务，将这些事务已提交的结果重新记入数据库

10.3.3 介质故障的恢复（4）

- 介质故障的恢复需要DBA介入
  - DBA的工作
    - 重装最近转储的数据库副本和有关的各日志文件副本
    - 执行系统提供的恢复命令
  - 具体的恢复操作仍由DBMS完成

10.4 恢复的实现技术（1）

- 恢复操作的基本原理：冗余
  - 利用存储在系统其它地方的冗余数据来重建数据库中已被破坏或不正确的那部分数据
- 恢复的实现技术：复杂
  - 一个大型数据库产品，恢复子系统的代码要占全部代码的10%以上

10.4 恢复的实现技术（2）

- 恢复机制涉及的关键问题
  - 1. 如何建立冗余数据
    - 数据转储（backup）
    - 登记日志文件（logging）
  - 2. 如何利用这些冗余数据实施数据库恢复

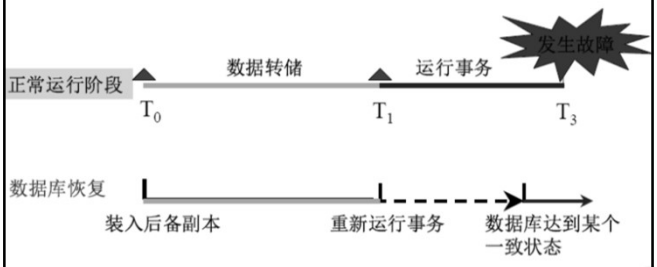
10.5 数据转储与恢复

- 10.5.1 什么是转储
- 10.5.2 转储的用途
- 10.5.3 转储方法

10.5.1 什么是转储

- 转储是指DBA将整个数据库复制到磁带或另一个磁盘上保存起来的过程，备用的数据称为后备副本或后援副本
- 如何使用
  - 数据库遭到破坏后可以将后备副本重新装入
  - 重装后备副本只能将数据库恢复到转储时的状态

10.5.2 转储的用途



10.5.3 转储方法

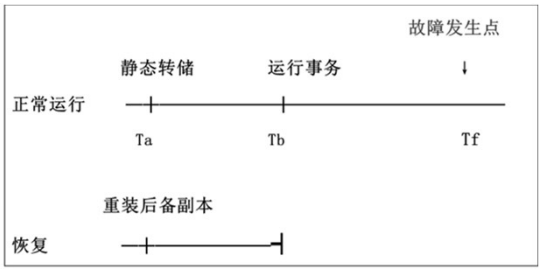
一、静态转储与动态转储
二、海量转储与增量转储
三、转储方法小结

一、静态转储与动态转储（1）

- 静态转储
  - 在系统中无运行事务时进行转储
  - 转储开始时数据库处于一致性状态
  - 转储期间不允许对数据库进行任何的存取、修改活动
  - 优点：实现简单
  - 缺点：降低了数据库的可用性
    - 转储必须等用户事务结束
    - 新的事务必须等转储结束

一、静态转储与动态转储（2）

- 利用静态转储副本进行恢复



一、静态转储与动态转储（3）

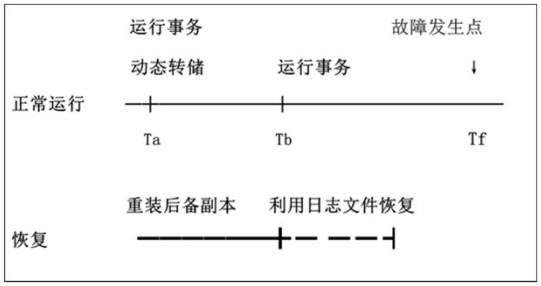
- 动态转储
  - 转储操作与用户事务并发进行
  - 转储期间允许对数据库进行存取或修改
  - 优点
    - 不用等待正在运行的用户事务结束
    - 不会影响新事务的运行
  - 动态转储的缺点
    - 不能保证副本中的数据正确有效

一、静态转储与动态转储（4）

- 利用动态转储得到的副本进行故障恢复
  - 需要把动态转储期间各事务对数据库的修改活动登记下来，建立日志文件
  - 后备副本加上日志文件才能把数据库恢复到某一时刻的正确状态

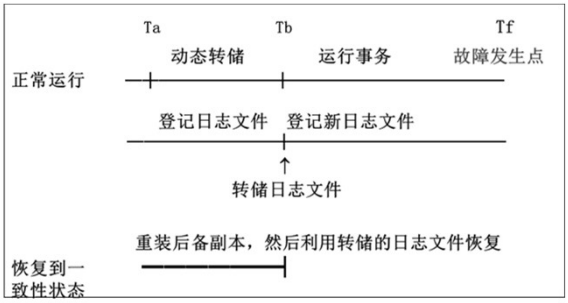
一、静态转储与动态转储（5）

- 利用动态转储副本进行恢复



一、静态转储与动态转储（6）

- 利用动态转储副本进行恢复（续）



二、海量转储与增量转储

- 海量转储：每次转储全部数据库
- 增量转储：只转储上次转储后更新过的数据
- 海量转储与增量转储比较
  - 从恢复角度看，使用海量转储得到的后备副本进行恢复往往更方便
  - 但如果数据库很大，事务处理又十分频繁，则增量转储方式更实用更有效

三、转储方法小结（1）

- 转储方法分类

		转储状态	
		动态转储	静态转储
转储方式	海量转储	动态海量转储	静态海量转储
	增量转储	动态增量转储	静态增量转储

三、转储方法小结（2）

- 转储策略
  - 应定期进行数据转储，制作后备副本
  - 但转储又是十分耗费时间和资源的，不能频繁进行
  - DBA应该根据数据库使用情况确定适当的转储周期和转储方法
    - 例
      - 每天晚上进行动态增量转储
      - 每周进行一次动态海量转储
      - 每月进行一次静态海量转储

10.6 基于日志的数据库恢复

- 10.6.1 数据库系统日志文件
- 10.6.2 使用日志恢复数据库

10.6.1 数据库系统日志文件

一、日志文件的格式和内容
二、日志文件的作用
三、登记日志文件

一、日志文件的格式和内容（1）

1. 什么是日志文件
   - 日志文件（log）是用来记录事务对数据库的更新操作的文件
2. 日志文件的格式
   - 以记录为单位的日志文件
   - 以数据块为单位的日志文件

一、日志文件的格式和内容（2）

3. 日志文件的内容
   - 各个事务的开始标记（BEGIN TRANSACTION）
   - 各个事务的结束标记（COMMIT或ROLLBACK）
   - 各个事务的所有更新操作
   - 与事务有关的内部更新操作
   - 日志文件中的一个日志记录 （log record）

一、日志文件的格式和内容（3）

4. 基于记录的日志文件
   - 每条日志记录的内容
     - 事务标识
     - 操作类型（插入、删除或修改）
     - 操作对象（记录ID、Block NO.）
     - 更新前数据的旧值（对插入操作而言，此项为空值）
     - 更新后数据的新值（对删除操作而言, 此项为空值）

一、日志文件的格式和内容（4）

5. 基于数据块的日志文件
   - 每条日志记录的内容
     - 事务标识（标明是哪个事务）
     - 操作对象（记录ID、Block NO.）
     - 更新前数据所在的整个数据块的值（对插入操作而言，此项为空值）
     - 更新后整个数据块的值（对删除操作而言, 此项为空值）

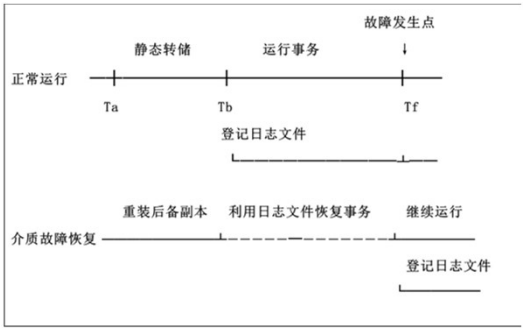
二、日志文件的作用（1）

1. 用途
   - 进行事务故障恢复
   - 进行系统故障恢复
   - 协助后备副本进行介质故障恢复

二、日志文件的作用（2）

2. 与静态转储后备副本配合进行介质故障恢复
   - 静态转储的数据已是一致性的数据
   - 如果静态转储完成后，仍能定期转储日志文件，则在出现介质故障重装数据副本后，可以利用这些日志文件副本对已完成的事务进行重做处理
   - 这样不必重新运行那些已完成的事务程序就可把数据库恢复到故障前某一时刻的正确状态

二、日志文件的作用（3）



二、日志文件的作用（4）

- 上图中
  - 系统在T_a时刻停止运行事务，进行数据库转储
  - 在T_b时刻转储完毕，得到T_b时刻的数据库一致性副本
  - 系统运行到T_f时刻发生故障
  - 为恢复数据库，首先由DBA重装数据库后备副本，将数据库恢复至T_b时刻的状态
  - 重新运行自T_b～T_f时刻的所有更新事务，把数据库恢复到故障发生前的一致状态

二、日志文件的作用（5）

3. 介质故障恢复：LOG FILE + 动态转储后备副本
  - 动态转储数据库：同时转储同一时间点的日志文件
  - 后备副本与该日志文件结合起来才能将数据库恢复到一致性状态
  - 利用这些日志文件副本进一步恢复事务，避免重新运行事务程序

三、登记日志文件（1）

- 为保证数据库是可恢复的，登记日志文件时必须遵循两条原则
  - 登记的次序严格按并行事务执行的时间次序
  - 必须先写日志文件，后写数据库
    - 写日志文件操作：把表示这个修改的日志记录写到日志文件
    - 写数据库操作：把对数据的修改写到数据库中

三、登记日志文件（2）

- 为什么要先写日志文件
  - 写数据库和写日志文件是两个不同的操作
  - 在这两个操作之间可能发生故障
  - 如果先写了数据库修改，而在日志文件中没有登记下这个修改，则以后就无法恢复这个修改了
  - 如果先写日志，但没有修改数据库，按日志文件恢复时只不过是多执行一次不必要的UNDO操作，并不会影响数据库的正确性

10.6.2 使用日志恢复数据库

- 基于日志的恢复技术
  - 一、Redo技术
  - 二、Undo恢复技术

一、Redo技术

- 在日志中记录所有数据库修改操作
- 将一个事务的写操作延迟到事务提交时进行
- 如果发生故障，可以用Redo操作重做事务，恢复已完成的事务

例题（1）

- 假设三个银行账户A、B和C的金额为1000、2000和3000元。执行T_0事务，将100元从账户A转到账户B

T_0

Read（A）；
A：=A-100；
Write（A）；
Read（B）；
B：=B+100；
Write（B）.

例题（2）

- 执行另一事务T_1，从账户C中取出200元

T_1

Read（C）；
C：=C-200；
Write（C）.

- 假设先执行T_0，然后执行T_1

例题（3）

- 与T_0和T_1对应的日志的两种状态

(a)	(b)
[start, T_0]	[start, T_0]
[T_0, A, 900]	[T_0, A, 900]
[T_0, B, 2100]	[T_0, B, 2100]
[commit, T_0]	[commit, T_0]
[start, T_1]	[start, T_1]
[T_1, C, 2800]	[T_1, C, 2800]
	[commit, T_1]

例题（4）

- 与T_0和T_1对应的日志和数据库的状态

日志	数据库
[start, T_0]	
[T_0, A, 900]	
[T_0, B, 2100]	
[commit, T_0]	
	A=900
	B=2100
[start, T_1]	
[T_1, C, 2800]	
[commit, T_1]	
	C=2800

例题（5）

- 两个事务T_0和T_1在运行，系统在事务完成前崩溃

（a）假设事务T_1中的Write（C）操作刚刚完成后发生了系统崩溃

（b）假设日志记录<commit T_1>刚写入日志后发生了系统崩溃

（a）

- 系统重新启动后，执行redo（T_0）操作，因为日志中有记录<commit T_0>。事务T_0提交后，账户A和B的值分别为900元和2100元，由于事务T_1没有提交，账户C的值仍为3000元。不完整的事务T_1的日志记录可以从日志中删除

[start, T_0]
[T_0, A, 900]
[T_0, B, 2100]
[commit, T_0]
[start, T_1]
[T_1, C, 2800]

（b）

- 系统重新启动后，日志中有两条提交记录：一个是T_0的，一个是T_1的。系统按照日志中记录的事务提交顺序执行redo（T_0）和redo（T_1）。这些操作执行后，账户A、B和C的值分别为900元、2100元和2800元

[start, T_0]
[T_0, A, 900]
[T_0, B, 2100]
[commit, T_0]
[start, T_1]
[T_1, C, 2800]

[commit, T_1]

二、Undo恢复技术（1）

- 在事务执行过程中修改了数据库而事务还没提交，此时如果系统崩溃，可以利用Undo恢复技术（撤销事务）将被修改的数据项恢复到事务开始前的状态

二、Undo恢复技术（2）

- Undo操作的过程
  - 检查日志文件，寻找事务T_i执行write（X）操作前写入日志的记录
  - 把数据库中的X项的值重新修改为更新前的旧值
  - 如果事务T_i有多个write操作，Undo write操作的顺序必须与write操作时写入日志的顺序相反

例题

- 与T_0和T_1对应的日志三种状态

(a)	(b)	(c)
[start, T_0]	[start, T_0]	[start, T_0]
[T_0, A, 1000, 900]	[T_0, A, 1000, 900]	[T_0, A, 1000, 900]
[T_0, B, 2000, 2100]	[T_0, B, 2000, 2100]	[T_0, B, 2000, 2100]
	[commit, T_0]	[commit, T_0]
	[T_1, start]	[T_1, start]
	[T_1, C, 3000, 2800]	[T_1, C, 3000, 2800]
		[commit, T_1]

（a）

- 事务T_0的操作write（B）刚刚执行完后系统发生崩溃
  - 系统重新启动后，检查日志中有记录<start T_0>，但没有记录<commit T_0>。T_0的操作必须撤销。系统通过执行undo（T_0），将账户A和账户B的值分别恢复成1000元和2000元

[start, T_0]
[T_0, A, 1000, 900]
[T_0, B, 2000, 2100]

（b）

- 事务T_1的操作write（C）刚刚执行完后就发生系统崩溃
  - 系统重新启动后，需要执行两个恢复动作，一个是undo（T_1），因为日志中只有记录<start T_1>。一个是redo（T_0），因为日志中包含了记录<commit T_0>。恢复过程完成后，账户A、B和C的数据分别为900、2100和3000元。

[start, T_0]
[T_0, A, 1000, 900]
[T_0, B, 2000, 2100]
[commit, T_0]
[T_1, start]
[T_1, C, 3000, 2800]

（c）

- 事务T_1提交后发生了系统崩溃
  - 系统重新启动后，事务T_0和T_1都需要重做，因为日志中含有记录<start T_0>和<commit T_0>以及<start T_1>和<commit T_1>。在系统执行恢复过程redo（T_0）和redo（T_1）后，账户A、B和C的值分别为900元、2100元和2800元

[start, T_0]
[T_0, A, 1000, 900]
[T_0, B, 2000, 2100]
[commit, T_0]
[T_1, start]
[T_1, C, 3000, 2800]
[commit, T_1]

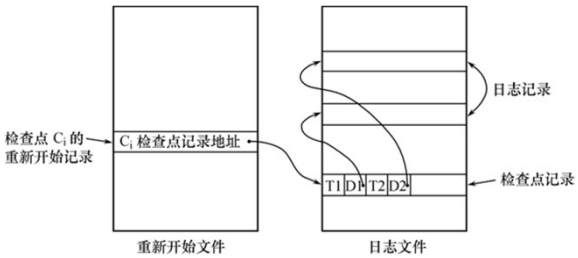
10.7 检查点恢复技术

- 10.7.1 问题的提出
- 10.7.2 检查点技术
- 10.7.3 利用检查点的恢复策略
- 10.7.4 利用检查点的恢复步骤

10.7.1 问题的提出（1）

- 两个问题
  - 搜索整个日志将耗费大量的时间
  - REDO处理：重新执行，浪费了大量时间
- 解决方案
  - 具有检查点（checkpoint）的恢复技术
    - 在日志文件中增加检查点记录（checkpoint）
    - 增加重新开始文件
    - 恢复子系统在登记日志文件期间动态地维护日志

10.7.1 问题的提出（2）



10.7.2 检查点技术（1）

- 检查点记录的内容
  1. 建立检查点时刻所有正在执行的事务清单
  2. 这些事务最近一个日志记录的地址
- 重新开始文件的内容
  - 记录各个检查点记录在日志文件中的地址

10.7.2 检查点技术（2）

- 在检查点维护日志文件
  1. 将当前日志缓冲区中的所有日志记录写入磁盘的日志文件上
  2. 在日志文件中写入一个检查点记录
  3. 将当前数据缓冲区的所有数据记录写入磁盘的数据库中
  4. 把检查点记录在日志文件中的地址写入一个重新开始文件

10.7.2 检查点技术（3）

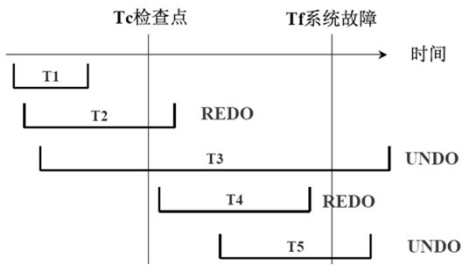
- 建立检查点
  - 定期
    - 按照预定的一个时间间隔
  - 不定期
    - 按照某种规则，如日志文件已写满一半建立一个检查点

10.7.3 利用检查点的恢复策略（1）

- 使用检查点方法可以改善恢复效率
  - 当事务T在一个检查点之前提交
    - T对数据库所做的修改已写入数据库
  - 写入时间是在这个检查点建立之前或在这个检查点建立之时
  - 在进行恢复处理时，没有必要对事务T执行REDO操作

10.7.3 利用检查点的恢复策略（2）

- 系统出现故障时，恢复子系统将根据事务的不同状态采取不同的恢复策略



10.7.3 利用检查点的恢复策略（3）

- T_1：在检查点之前提交
- T_2：在检查点之前开始执行，在检查点之后故障点之前提交
- T_3：在检查点之前开始执行，在故障点时还未完成
- T_4：在检查点之后开始执行，在故障点之前提交
- T_5：在检查点之后开始执行，在故障点时还未完成

10.7.3 利用检查点的恢复策略（4）

- 恢复策略
  - T_3和T_5在故障发生时还未完成，所以予以撤销
  - T_2和T_4在检查点之后才提交，它们对数据库所做的修改在故障发生时可能还在缓冲区中，尚未写入数据库，所以要REDO
  - T_1在检查点之前已提交，所以不必执行REDO操作

10.7.4 利用检查点的恢复步骤（1）

1. 从重新开始文件中找到最后一个检查点记录在日志文件中的地址，由该地址在日志文件中找到最后一个检查点记录

10.7.4 利用检查点的恢复步骤（2）

2. 由该检查点记录得到检查点建立时刻所有正在执行的事务清单ACTIVE-LIST
   - 建立两个事务队列
     - UNDO-LIST
     - REDO-LIST
   - 把ACTIVE-LIST暂时放入UNDO-LIST队列，REDO队列暂为空

10.7.4 利用检查点的恢复步骤（3）

3. 从检查点开始正向扫描日志文件，直到日志文件结束
   - 如有新开始的事务T_i，把T_i暂时放入UNDO-LIST队列
   - 如有提交的事务T_j，把T_j从UNDO-LIST队列移到REDO-LIST队列
4. 对UNDO-LIST中的每个事务执行UNDO操作，对REDO-LIST中的每个事务执行REDO操作

10.8 数据库镜像恢复技术（1）

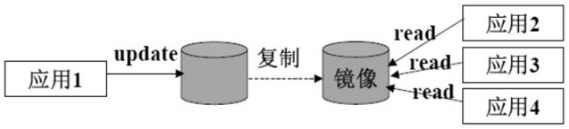
- 介质故障是对系统影响最为严重的一种故障，严重影响数据库的可用性
  - 介质故障恢复比较费时
  - 为预防介质故障，DBA必须周期性地转储数据库

10.8 数据库镜像恢复技术（2）

- 提高数据库可用性的解决方案
  - 数据库镜像（Mirror）
    - 数据库镜像是指在不同的设备上同时存在两份数据库，一个是主设备，另一个是镜像设备
    - 所谓镜像的含义是指每当主设备的数据库发生更新时，系统自动更新镜像设备的数据，使得两个设备的数据始终保持一致

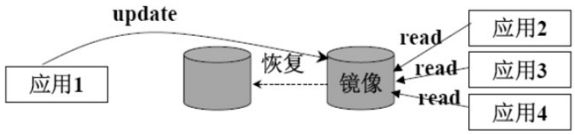
10.8 数据库镜像恢复技术（3）

- 数据库镜像
  - DBMS自动把整个数据库或其中的关键数据复制到另一个磁盘上
  - DBMS自动保证镜像数据与主数据的一致性
  - 每当主数据库更新时，DBMS自动把更新后的数据复制过去



10.8 数据库镜像恢复技术（4）

- 数据库镜像的用途
  - 出现介质故障时
    - 可由镜像磁盘继续提供使用
    - 同时DBMS自动利用镜像磁盘数据进行数据库的恢复
    - 不需要关闭系统和重装数据库副本



10.8 数据库镜像恢复技术（5）

- 数据库镜像的用途（续）
  - 没有出现故障时
    - 可用于并发操作
    - 一个用户对数据加排他锁修改数据
    - 其他用户可以读镜像数据库上的数据，而不必等待该用户释放锁
- 频繁地复制数据自然会降低系统运行效率
  - 在实际应用中用户往往只选择对关键数据和日志文件镜像，而不是对整个数据库进行镜像

10.9 小结（1）

- 如果数据库只包含成功事务提交的结果，就说数据库处于一致性状态。保证数据一致性是对数据库的最基本的要求
- 事务是数据库的逻辑工作单位
  - 事务的四个特征
    - 原子性（Atomicity）
    - 一致性（Consistency）
    - 隔离性（Isolation）
    - 持久性（Durability）

10.9 小结（2）

- DBMS必须对事务故障、系统故障和介质故障进行恢复
- 恢复中最经常使用的技术：数据库转储和登记日志文件
- 恢复的基本原理：利用存储在后备副本、日志文件和数据库镜像中的冗余数据来重建数据库

10.9 小结（3）

- 常用恢复技术
  - 事务故障的恢复
    - UNDO
  - 系统故障的恢复
    - UNDO + REDO
  - 介质故障的恢复
    - 重装备份并恢复到一致性状态 + REDO

10.9 小结（4）

- 提高恢复效率的技术
  - 检查点技术
    - 可以提高系统故障的恢复效率
    - 可以在一定程度上提高利用动态转储备份进行介质故障恢复的效率
  - 镜像技术
    - 镜像技术可以改善介质故障的恢复效率

参考资料

- 第8章，周志逵 郭贵锁等编著，数据库系统原理，北京：清华大学出版社，2008
- 第十章，王珊 萨师煊，数据库系统概论（第5版）， 北京：高等教育出版社，2014
- 第8章，苗雪兰 刘瑞新 王怀峰编著，数据库系统原理及应用教程，机械工业出版社，2003

Thank You